

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

RecyclerView y Adapters



Universidad
Rey Juan Carlos

RecyclerView

Cuando el contenido de tu diseño no sea fijo (por ejemplo, tu lista de contactos), se puede usar un *RecyclerView*. Este *ViewGroup* facilita la gestión de vistas que se modifican durante el tiempo de ejecución.

RecyclerView permite mostrar de manera eficiente grandes conjuntos de datos. Tú proporcionas los **datos** y defines el **aspecto** de cada elemento, y la biblioteca *RecyclerView* creará los **elementos de forma dinámica** cuando se los necesite.

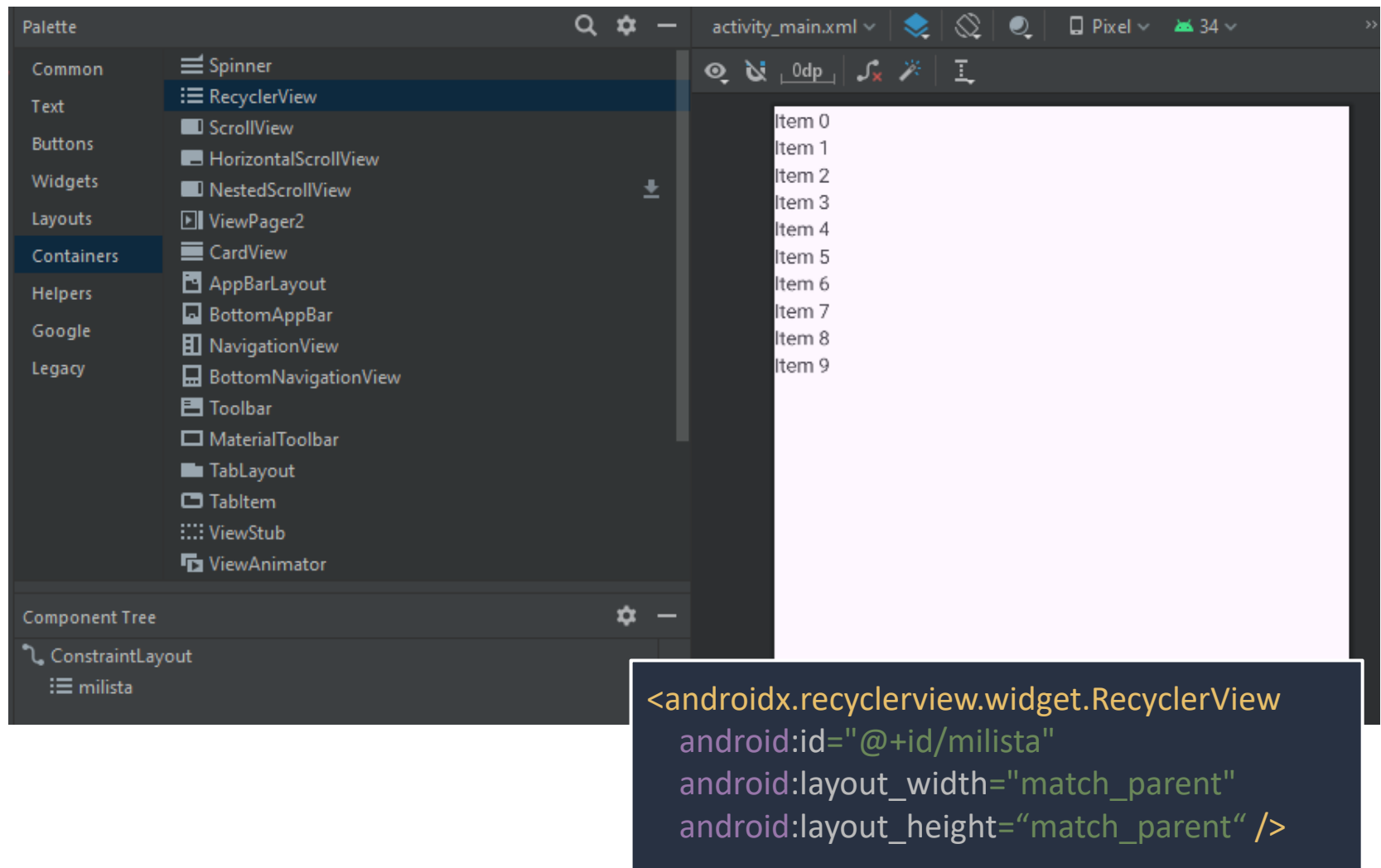


Componentes clave

Varias clases trabajan juntas para compilar tu lista dinámica.

- **RecyclerView** es el ViewGroup que contiene las vistas correspondientes a tus datos. Se puede agregar al layout desde el editor de diseños (*Containers*).
- **Contenedor de vistas:** representa el diseño de cada elemento que se va a mostrar. Contiene atributos que referencian todas las vistas que componen de su layout. Para definirlo: extiende *RecyclerView.ViewHolder*.
- **Adaptador:** es el encargado de crear las vistas de cada elemento y rellenarlo con los datos correspondientes. Para definirlo: extiende *RecyclerView.Adapter*.
- **Administrador de diseño:** organiza los elementos de tu lista. Se puede utilizar *LinearLayoutManager*, *GridLayoutManager*, o definir uno propio.

Añadir el RecyclerView al diseño



Definir el diseño de cada item

En un archivo .xml aparte dentro de *res/layout*, se diseña el aspecto de la vista que tendrá cada elemento de nuestra lista/cuadrícula.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="TextView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Item_lista.xml

Crear el contenedor de vistas (ViewHolder)

Es una clase que se utiliza para representar y manipular la vista de cada elemento que se va a mostrar.

- Hereda de *RecyclerView.ViewHolder*
- Recibe una *View* en su constructor, que es la que contiene el diseño del elemento.
- Debe definir un atributo para cada vista del diseño, para posteriormente permitir al adaptador mostrar los datos correspondientes.

```
class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {  
    val textView = row.findViewById<TextView>(R.id.textView)  
}
```

MyViewHolder.kt

Crear el adaptador

Suministra los datos y diseños que muestra el RecyclerView. Un Adapter personalizado extiende de *RecyclerView.Adapter* y sobrescribe estas tres funciones:

onCreateViewHolder(): RecyclerView llama a este método siempre que necesita crear una *ViewHolder* nueva. El método crea y, luego, inicializa la ViewHolder y su View asociada, pero no completa el contenido de la vista.

onBindViewHolder(): RecyclerView llama a este método para asociar una *ViewHolder* con los datos. El método recupera los datos correspondientes y los usa para completar el diseño del contenedor de vistas.

getItemCount(): RecyclerView llama a este método para obtener el tamaño del conjunto de datos. Se utiliza internamente para saber cuando no hay más elementos que se puedan mostrar.

Crear el adaptador (*cont.*)

Ejemplo de un Adapter propio cuya fuente de datos es una lista de String.

onCreateViewHolder crea la vista mediante el ViewHolder con el diseño de un elemento. *onBindViewHolder* rellena la vista con los datos de la posición correspondiente.

MyAdapter.kt

```
class MyAdapter(val data: List<String>) : RecyclerView.Adapter<MyViewHolder>() {  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {  
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_lista,  
            parent, false)  
        return MyViewHolder(layout)  
    }  
  
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {  
        holder.textView.text = data[position]  
    }  
  
    override fun getItemCount(): Int = data.size  
}
```

Devuelve el número de
elementos

Configurar el RecyclerView en la actividad

1. Inicializar la variable que referencia al RecyclerView en el layout.
2. Establecer un LayoutManager en él para indicar como mostrar los elementos.
3. Instanciar el adaptador con los datos y asignárselo al RecyclerView.

```
class MainActivity : AppCompatActivity() {  
    private lateinit var miLista: RecyclerView  
    private val data = mutableListOf<String>()  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        miLista = findViewById(R.id.milista)  
        miLista.layoutManager = LinearLayoutManager(this)  
        // LinearLayoutManager(this).apply { orientation=LinearLayoutManager.HORIZONTAL }  
        var miAdaptador = MyAdapter(data)  
        miLista.adapter = miAdaptador  
    }  
}
```

MainActivity.kt

Ejercicio 1

1. Crea una aplicación nueva con una única actividad
2. En el diseño, incluye un botón y un RecyclerView que ocupe el resto de la pantalla
3. Crea las clases y diseños necesarios para mostrar elementos que contengan una String en mayúsculas y en negrita.
4. Añade la lógica necesaria para que al pulsar el botón se añada un nuevo elemento a la lista dinámica mostrada en el RecyclerView

Para refrescar el RecyclerView tras modificar los datos hay varias opciones

```
miAdaptador.notifyDataSetChanged() // No recomendada, reconstruye todo de cero  
miAdaptador.notifyItemInserted(data.size) // Más rápida. Específica para insertar
```

Otras: notifyItemChanged, notifyItemRemoved, notifyItemRangeChanged, notifyItemRangeInserted, notifyItemRangeRemoved

RecyclerView con datos propios

Clase modelo

Cuando queremos mostrar algo más que un dato simple por cada elemento de la lista, se deben **utilizar datos propios**. Para ello, creamos **una *clase modelo***.

Esta clase modelo, típicamente una *data class*, recoge todos los atributos del elemento que se quiere mostrar (y quizás algunos que no).

Ejemplo:

Para una aplicación donde queremos mostrar una lista de la compra, cada elemento puede contener el nombre del producto y las unidades a adquirir.

```
data class ItemCompra(val producto: String, val cantidad: Int) : Serializable
```

ItemCompra.kt



Permite meterlo como extra en un Intent.

Ejercicio 2

1. Crea una aplicación nueva con una única actividad para guardar contactos en una lista.
2. En el diseño, abajo, incluye dos campos de texto (para el nombre y el teléfono) y un botón. Además, añade un RecyclerView que ocupe el resto de la pantalla
3. Crea las clases y diseños necesarios para mostrar, para cada elemento, el nombre y el teléfono de un contacto.
4. Añade la lógica necesaria para que al pulsar el botón se añada un nuevo contacto a la lista dinámica mostrada en el RecyclerView

Eventos en un RecyclerView

Interfaz de evento en el adaptador

Varias clases trabajan juntas para compilar tu lista dinámica.

```
interface RecyclerViewEvent {  
    fun onItemClick(position: Int)  
}
```

1. Definimos una interfaz con un método por evento

MainActivity.kt

2. Añadimos la interfaz como atributo al constructor

```
class MyAdapter(val data: List<String>, val listener: RecyclerViewEvent? = null) :  
    RecyclerView.Adapter<MyAdapter.MyViewHolder>() {  
    inner class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row), View.OnClickListener {  
        val textView = row.findViewById<TextView>(R.id.textView)  
        init {  
            row.setOnClickListener(this)  
        }  
  
        override fun onClick(p0: View?) {  
            val position = adapterPosition  
            if (position != RecyclerView.NO_POSITION) {  
                listener?.onItemClick(position)  
            }  
        }  
    }  
    // ... onCreateViewHolder y demás funciones  
}
```

3. El *ViewHolder* lo ponemos como una *inner class* que implementa *View.OnClickListener*

4. Asignamos el *callback* a la vista entera, o a uno de sus elementos

5. Implementamos el método *onClick*, donde llamamos a la función recibida por el constructor

Asignar función en la actividad

Al instanciar el Adapter, se puede pasar un segundo parámetro opcional al constructor con un objeto que implemente la interfaz definida para asignarle una función al evento *onClick* del elemento.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
    miLista = findViewById(R.id.milista)  
    miLista.layoutManager = LinearLayoutManager(this)  
    var miAdaptador = MyAdapter(data, object : RecyclerViewEvent {  
        override fun onItemClick(position: Int) {  
            TODO("Implementar la funcion a ejecutar cuando se haga click")  
        }  
    })  
    miLista.adapter = miAdaptador  
}
```

MainActivity.kt

Ejercicio 3

Siguiendo con el ejercicio anterior de la lista de contactos:

1. Añade dos campos de entrada más al cuadro inferior, uno para recoger el email y otro para la fecha de cumpleaños.
2. Modifica la clase modelo para que almacene todos los atributos de un contacto.
3. Al pulsar el botón, se añadirá el contacto a la lista. El diseño de la vista de cada elemento en la lista no varía (muestra solo nombre y teléfono).
4. Crea una segunda actividad *DetallesContactoActivity* donde se muestren todos los atributos de un contacto (nombre, número, email y cumpleaños).
5. Modifica la aplicación para que, al hacer click en un contacto de la lista, se lance la actividad con la información detallada del mismo.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>